Contenido

1.Funciones print e input 2

2. Operaciones aritméticas 2

2.1. Operaciones básicas 2

2.2. Otras funciones integradas 4

2.3. Valores aleatorios 5

2.4. Operadores 5

3. Listado de ejercicios operaciones básicas y E/S teclado 5

4. if y if/else 6

4.1. if 6

4.2.if/else 7

4.3.Operadores lógicos 8

5.Listado de ejercicios if/else 8

6.Bucle for 9

6.1. Tipos de datos utilizados con bucles 9

6.2. Función range() 9

6.3. Sintaxis bucle for 10

7. Ejercicios bucle for 10

8. Bucle while 11

9.Ejercicios bucle while 11

GUÍA DE PYTHON

Para ejecutar el código podéis utilizar este editor online:

https://repl.it/languages/python3

1.Funciones print e input

- **print:** Se utiliza para mostrar por pantalla un mensaje, por ejemplo:

  print(“Hola Mundo”)

- **input**: Se utiliza para obtener el valor pasado por teclado. Ejemplo:

  print(“¿Cómo te llamas”)

  nombre = input()

Para mostrar por pantalla el nombre de una variable, en este caso la variable nombre, se debe poner entre corchetes:

print(f"Me alegro de conocerle, **{nombre}**")

Hay que tener en cuenta, que al pasar por teclado cualquier valor lo va a interpretar como una cadena, si no queremos que sea una cadena hay que convertirlo.

Ejemplo:

Print(“Díme tu edad:”)

edad = **int**(input())

2. Operaciones aritméticas

2.1. Operaciones básicas

Tipos de datos numéricos: enteros (**int**) y decimales (**float**)

Python distingue entre números enteros y decimales. Al escribir un número decimal, el **separador** entre la parte entera y la parte decimal es un **punto**.

- **Las cuatro operaciones aritméticas básicas son la suma (+), la resta (-), la multiplicación (*) y la división (/).**

  Cuando en una fórmula aparecen varias operaciones, Python las efectúa aplicando las reglas usuales de prioridad de las operaciones (primero multiplicaciones y divisiones, después sumas y restas).

  Ejemplo:

```
>>> 1 + 2 * 3
7
```

```
>>> 10 - 4 * 2
2
```

En caso de querer que las operaciones se realicen en otro orden, se deben utilizar paréntesis.

```
>>> (5 + 8) / (7 - 2)
2.6
```

- **Cociente de una división**

El cociente de una división se calcula en Python con el operador //.

Ejemplo:

```
>>> 10 // 3
3
```

- **Resto de una división**

El resto de una división se calcula en Python con el operador %

Ejemplo:

```
>>> 10 % 3
1
```

- **Potencias y raíces**

Las potencias se calculan con el **operador ****, teniendo en cuenta que x ** y = xy.

Las potencias tienen prioridad sobre las multiplicaciones y divisiones.

Ejemplo:

```
>>> 2 ** 3
8
```

También se puede calcular potencias con la función **pow**(x,y)

```
>>> pow(2, 3)
8
```

- **Redondeo de un número**

Para redondear un número se utiliza la función **round().**

Si sólo hay un argumento, la función devuelve el argumento redondeado al entero más próximo:

```
>>> round(4.35)
4
```

Si se escriben dos argumentos, siendo el segundo un número entero, la función integrada round() devuelve el primer argumento redondeado en la posición indicada por el segundo argumento.

Si el segundo argumento es positivo, el primer argumento se redondea con el número de decimales indicado:

```
>>> round(4.3527, 2)
4.35
```

2.2. Otras funciones integradas

- **Valor absoluto:** abs()

  La función integrada abs() calcula el valor absoluto de un número, es decir, el valor sin signo.

```
>>> abs(-6)
6
>>> abs(7)
7
```

- **Máximo: max()**

  La función integrada max() calcula el valor máximo de un conjunto de valores (numéricos o alfabéticos). En el caso de cadenas, el valor máximo corresponde al último valor en orden alfabético, sin importar la longitud de la cadena.

```
>>> max(4, 5, -2, 8, 3.5, -10)
8
>>> max("David", "Alicia", "Tomás", "Emilio")
'Tomás'
```

  Las vocales acentuadas, la letra ñ o ç se consideran posteriores al resto de vocales y consonantes

```
>>> max("Ángeles", "Roberto")
'Ángeles'
```

- **Mínimo: min()**

  La función integrada min() calcula el valor mínimo de un conjunto de valores (numéricos o alfabéticos). En el caso de cadenas, el valor mínimo corresponde al primer valor en orden alfabético, sin importar la longitud de la cadena.

```
>>> min(4, 5, -2, 8, 3.5, -10)
-10
```

```
>>> min("David", "Alicia", "Tomás", "Emilio")
'Alicia'
```

Las vocales acentuadas, la letra ñ o ç se consideran posteriores al resto de vocales y consonantes

```
>>> min("Ángeles", "Roberto")
'Roberto'
```

2.3. Valores aleatorios

- **Random()**
  La biblioteca **random** contiene una serie de funciones relacionadas con los valores aleatorios.
  Como ocurre con todas las bibliotecas, antes de utilizar una función se debe importar la función o la biblioteca entera.
  La función **randrange** devuelve un número aleatorio entre 0 y el número pasado
  Ejemplo:
  import random
  print(random.randrange(10))

2.4. Operadores

Con los valores numéricos se pueden utilizar operadores tales como

- > : mayor que
- < : menor que
- <= y >= : mayor/menor o igual que
- !=: distinto

3. Listado de ejercicios operaciones básicas y E/S teclado

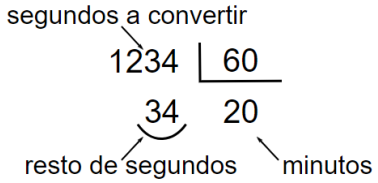
Ejemplo: Aquí hay un ejemplo de como sería la media de dos números

```
print('Introduce un numero')
numero1=input()
print("Escribe otro numero")
numero2=input()
media=(int(numero1)+int(numero2))/2
print(f"La media es: {media}" )
```

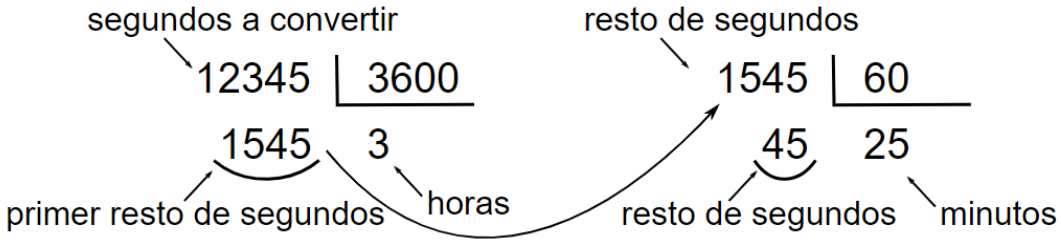
Resultado:

```
Introduce un numero
6
Escribe otro numero
8
La media es: 7.0
```

1. Escriba un programa que pida tres números y que escriba su media aritmética. Se recuerda que la media aritmética de tres números es la suma de todos los números dividida por 3.
2. Escriba un programa que pida el peso (en kilogramos) y la altura (en metros) de una persona y que calcule su índice de masa corporal (imc).
   Se recuerda que el imc se calcula con la fórmula $imc = peso/altura^2$
3. Escriba un programa que pida una distancia en pulgadas y que escriba esa distancia en centímetros.
   Se recuerda que una pulgada son 2,54 cm.
4. Escriba un programa que pida una distancia en pies y que escriba esa distancia en centímetros.
   Se recuerda que un pie son doce pulgadas y una pulgada son 2,54 cm.
5. Escriba un programa que pida una temperatura en grados Celsius y que escriba esa temperatura en grados Fahrenheit.
   Se recuerda que la relación entre grados Celsius (C) y grados Fahrenheit (F) es la siguiente: F = 1,8 * C + 32
6. Escriba un programa que pida una temperatura en grados Fahrenheit y que escriba esa temperatura en grados Celsius.
   Se recuerda que la relación entre grados Celsius (C) y grados Fahrenheit (F) es la siguiente: C = (F - 32) / 1,8
7. Escriba un programa que pida una cantidad de segundos y que escriba cuántos minutos y segundos son.
   Ayuda:



8. Escriba un programa que pida una cantidad de segundos y que escriba cuántas horas, minutos y segundos son
   Ayuda:



4. if y if/else

4.1. if

La sentencia if permite que un conjunto de instrucciones se ejecute si se cumple la condición.

El resultado de la condición siempre es un valor booleano (true o false)

```
if condición:
    aquí van las órdenes que se ejecutan si la condición es cierta
    y que pueden ocupar varias líneas
```

La ejecución de esta construcción es la siguiente:

1. La condición se evalúa siempre.
2. Si el resultado es True se ejecuta el bloque de sentencias
3. Si el resultado es False no se ejecuta el bloque de sentencias.
4. La primera línea contiene la condición a evaluar y es una expresión lógica. Esta línea debe terminar siempre por dos puntos (:).

A continuación, viene el bloque de órdenes que se ejecutan cuando la condición se cumple (es decir, cuando la condición es verdadera). Es importante señalar que **este bloque debe ir sangrado**, puesto que Python utiliza el sangrado para reconocer las líneas que forman un bloque de instrucciones. El sangrado que se suele utilizar en Python es de **cuatro espacios**, pero se pueden utilizar más o menos espacios. Al escribir dos puntos (:) al final de una línea, el editor sangrará automáticamente las líneas siguientes. **Para terminar un bloque, basta con volver al principio de la línea.**

Ejemplo:

```
print('Escribe un numero')
numero=input()
if int(numero) <5:
    print("El número debería ser mayor que 5")
```

4.2. if/else

La sintaxis de la construcción if ... else ... es la siguiente:

```
if condición:
    aquí van las órdenes que se ejecutan si la condición es cierta
    y que pueden ocupar varias líneas
else:
    y aquí van las órdenes que se ejecutan si la condición es
    falsa y que también pueden ocupar varias líneas
```

La ejecución de esta construcción es la siguiente:

- La condición se evalúa siempre.
- Si el resultado es True se ejecuta solamente el bloque de sentencias 1
- Si el resultado es False se ejecuta solamente el bloque de sentencias 2.

Ejemplo:

```
print('Escribe un numero')
numero=input()
if int(numero) <5:
    print("El número debería ser mayor que 5")
else:
    print("Tu número es mayor que cero")
```

4.3. Operadores lógicos

Son unas operaciones que trabajan con valores booleanos.

- **and**: "y" lógico. Este operador da como resultado True **si y sólo si** sus dos operandos son True
- **or**: "o" lógico. Este operador da como resultado True **si algún operando** es True

5.Listado de ejercicios if/else

1. Escriba un programa que pida primero un número par y luego un número impar (positivos o negativos). En caso de que uno o los dos valores no sea correcto, se mostrará un único aviso.
2. Escriba un programa que pida primero un número par (positivo o negativo) y si el valor no es correcto, muestre un aviso. Si el valor es correcto, pedirá un número impar (positivo o negativo) y si el valor no es correcto, mostrará un aviso.
3. Escriba un programa que pida primero un número par y a continuación un numero impar (positivos o negativos). En caso de que uno o los dos valores no sean correctos, se mostrarán uno o dos avisos.
4. Escriba un programa que pida primero un número par y a continuación un numero impar (positivos o negativos). En cada petición, si el valor no es correcto se mostrará un aviso.
5. Escriba un programa que pida dos números enteros y que calcule su división, escribiendo si la división es exacta o no.
6. Mejore el programa anterior haciendo que tenga en cuenta que no se puede dividir por cero
7. Escriba un programa que pida dos números y que conteste cuál es el menor y cuál el mayor o que escriba que son iguales.
8. Escriba un programa que pida el año actual y un año cualquiera y que escriba cuántos años han pasado desde ese año o cuántos años faltan para llegar a ese año
9. Escriba un programa que pida dos números enteros y que escriba si el mayor es múltiplo del menor.
10. Mejore el programa anterior haciendo que el programa avise cuando se escriben valores negativos o nulos.

6.Bucle for

6.1. Tipos de datos utilizados con bucles

Los tipos de datos utilizados en los bucles, son aquellos datos compuestos, es decir, aquellos que contienen más de un valor.

En Python se utilizan las listas (arrays) y tuplas entre otros objetos.

- Listas (array)

  Las listas son conjuntos ordenados de elementos (números, cadenas, listas, etc). Las listas se delimitan por corchetes ([ ]) y los elementos se separan por comas.

  Ejemplo:

```
>>> primos = [2, 3, 5, 7, 11, 13]
>>> diasLaborables = ["Lunes", "Martes", "Miércoles", "Jueves", "Viernes"]
```

- Tuplas
  Es similar a la lista, pero sus valores son inmutables, no se pueden cambiar a lo largo del desarrollo del programa.

Para acceder a una posición de la lista es suficiente con nombrar la lista y la posición entre [ ].

Ejemplo:

```
>>> primos = [2, 3, 5, 7, 11, 13]
p=primos[1]
print(f"El numero primo es: {p}")
"El numero primo es 3"
```

6.2. Función range()

La función range() es una función integrada en Python que crea una secuencia ordenada de enteros que podemos utilizar para crear iteraciones. Por ello la función range() acepta tres números enteros de los cuales dos son opcionales.

range(inicio, fin, paso)

- Inicio: es el valor inicial de la secuencia (0 si no se especifica).
- Fin: es el valor final de la secuencia, el cual no se incluye en el resultado.
- Paso: indica el incremento entre elementos de la secuencia (1 si no se especifica).

Ejemplo:



6.3. Sintaxis bucle for

En general, un bucle es una estructura de control que repite un bloque de instrucciones. Un bucle for es un bucle que repite el bloque de instrucciones un número prederminado de veces. El bloque de instrucciones que se repite se suele llamar cuerpo del bucle y cada repetición se suele llamar iteración.

La sintaxis de un bucle for es la siguiente:

```
for variable in elemento iterable (lista, cadena, range, etc.):
    cuerpo del bucle
```

El cuerpo del bucle se ejecuta tantas veces como elementos tenga el elemento iterable.

Ejemplo de bucle 1

```
print("Comienzo")
for i in [0, 1, 2]:
    print("Hola ", end="")
print()
print("Final")
```

```
Comienzo
Hola Hola Hola
Final
```

```
print("Comienzo")
for i in [3, 4, 5]:
    print(f"Hola. Ahora i vale {i} y su cuadrado {i ** 2}")
print("Final")
```

```
Comienzo
Hola. Ahora i vale 3 y su cuadrado 9
Hola. Ahora i vale 4 y su cuadrado 16
Hola. Ahora i vale 5 y su cuadrado 25
Final
```

7. Ejercicios bucle for

1. Escribe un programa que pida un número e imprima por pantalla la tabla de multiplicar de ese número.

2. Crear una lista como esta numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14] y que muestre por pantalla cuales de los números son pares.

3. Utilizando la lista anterior, haced que muestre por pantalla cuales de los números son impares.

3. Escribir un programa que pida al usuario una palabra y la muestre por pantalla 10 veces.

4. Escribir un programa que pida al usuario un número entero positivo y muestre por pantalla todos los números impares desde 1 hasta ese número separados por comas.

5. Escribir un programa que pida al usuario un número entero positivo y muestre por pantalla la cuenta atrás desde ese número hasta cero separados por comas.

6. Escribir un programa que me muestre el número y su cuadrado del 1 al 6.

7. Escribir un programa que cuente del 1 al 10. Escriba los valores en pantalla. Si el número contado es 3, debe imprimir el siguiente mensaje “ha llegado al número 3”. Si el número contado es 10 imprima: “He terminado de contar”

8. Bucle while

Un bucle while permite repetir la ejecución de un grupo de instrucciones mientras se cumpla una condición (es decir, mientras la condición tenga el valor True).

La sintaxis del bucle while es la siguiente:

```
while condicion:
    cuerpo del bucle
```

La ejecución de esta estructura de control while es la siguiente:

Python evalúa la condición:

si el resultado es True se ejecuta el cuerpo del bucle. Una vez ejecutado el cuerpo del bucle, se repite el proceso (se evalúa de nuevo la condición y, si es cierta, se ejecuta de nuevo el cuerpo del bucle) una y otra vez mientras la condición sea cierta.

si el resultado es False, el cuerpo del bucle no se ejecuta y continúa la ejecución del resto del programa.

La variable o las variables que aparezcan en la condición se suelen llamar variables de control. Las variables de control deben definirse antes del bucle while y modificarse en el bucle while.

Ejemplo:

```
i = 1
while i <= 3:
    print(i)
    i += 1
print("Programa terminado")
```

```
1
2
3
Programa terminado
```

Hay que tener cuidado cuando se programa un bucle while, que no sea un **bucle infinito,** ya que así se conseguirá que nunca termine el bucle y el programa explote.

Es fácil cometer ciertos errores y programar un bucle infinito. Alguno de los errores más comunes son:

- El programador ha olvidado modificar la variable de control dentro del bucle
- El programador ha escrito una condición que se cumplirá siempre

9.Ejercicios bucle while

1. Escriba un programa que pida dos números enteros. El programa pedirá de nuevo el segundo número mientras no sea mayor que el primero. El programa terminará escribiendo los dos números.

2. Escriba un programa que pida números decimales mientras el usuario escriba número mayores que el primero.

3. Escriba un programa que pida números enteros mientras sean cada vez más grandes.

4. Escriba un programa que pida la cantidad de números positivos que se tienen que escribir y a continuación pida números hasta que se haya escrito la cantidad de números positivos indicada.

5. Escriba un programa que pida números mientras no se escriba un número negativo. El programa terminará escribiendo la suma de los números introducidos.

6. Escriba un programa que pida un valor límite positivo y a continuación pida números hasta que la suma de los números introducidos supere el límite inicial.

7. Escriba un programa que pida primero dos números enteros (mínimo y máximo) y que después pida números enteros situados entre ellos. El programa terminará cuando se escriba un número que no esté comprendido entre los dos valores iniciales. El programa termina escribiendo la cantidad de números escritos.

8. Escriba un programa que pida números pares mientras el usuario indique que quiere seguir introduciendo números. Para indicar que quiere seguir escribiendo números, el usuario deberá contestar S o s a la pregunta.